Point Receivers

Because Go strictly uses **Call by Value**, this default behavior creates two major problems when building Object-Oriented programs.

1. Problem 1: You Cannot Modify the Original Object

If you attach a method to a standard struct (a **Value Receiver**), Go makes a complete, independent copy of your object and hands it to the method.

- If you write a method and try to update **x**, it will successfully update the **x** coordinate, but only on the copy.

```
func (p Point) OffsetX(v float64) {
    p.x = p.x + v
}
```

- As soon as the method finishes, the copy is destroyed, and your original **Point** remains completely unchanged.

2. Problem 2: Massive Performance Hits

If your object is huge, copying it becomes a severe performance bottleneck.

- If you have an Image struct containing a **100x100** array of pixels (**10,000 integers**) and you call myImage.Blur(), Go has to duplicate all **10,000 integers** in memory just to execute the method.
- If you call that method 60 times a second for an animation, your program will grind to a halt.

3. The Solution: Pointer Receivers (*Type)

To solve both of these problems, you simply change the receiver type to be a **Pointer**.

- **The Syntax:** You add an asterisk * in front of the type name in the receiver block.

```
func (p *Point) OffsetX(v float64) {
    p.x = p.x + v
}
```

- **How it works:** Now, when you call **p1.OffsetX(5),** Go does not copy the entire Point struct. Instead, it implicitly passes a tiny, lightweight memory address (the pointer) that points directly to your original p1 object.
- **The Result:**
    1. Because the method has the memory address, it successfully alters the original object!
    2. Because a pointer is just a tiny 8-byte address, you completely avoid the performance penalty of copying large structs like images.

Point Receivers, Referencing, Dereferencing

When working with pointers, you normally have to manually use the **& (address-of)** operator to get a memory location, and the *** (dereference)** operator to read the actual data. When it comes to **Methods**, Go handles all of this math for you **automatically** behind the scenes.

1. Automatic Dereferencing (Inside the Method)

If you define a method with a pointer receiver, you do not need to manually dereference the pointer to interact with the **struct's** fields.

- **The Hard Way (C/C++ style): (*p).x = (*p).x + v** (You tell the computer to follow the pointer to the data, grab the x value, and update it).
- **The Go Way:** p.x = p.x + v
- **Why it works:** Because the receiver is (p *Point), the Go compiler already knows p is a pointer. It automatically applies the * under the hood whenever you type p.x.

2. Automatic Referencing (Calling the Method)

The exact same convenience applies when you call the method from your main function.

- **The Setup:** You create a normal, value-based object: **p := Point{3, 4}.** Note that p is not a pointer yet.
- **The Hard Way:** Because your OffsetX method expects a pointer, you would normally have to pass the memory address manually: (&p).OffsetX(5).
- **The Go Way:** You can just write **p.OffsetX(5).**
- **Why it works:** Go looks at the method signature, sees it requires a pointer, and silently converts p to &p for you during execution.

3. The Golden Rule: Consistency is Key

Because Go does all this referencing and dereferencing invisibly, the syntax looks exactly the same whether a method uses a Value Receiver or a Pointer Receiver.

This leads to a strict Go community best practice: **Do not mix and match receiver types for a single struct.**

- If a struct has five methods, they should all be Value Receivers, or they should all be Pointer Receivers.
- If you mix them, you (or other developers) will easily lose track of which methods are safely copying the data and which ones are actively mutating the original object, leading to extremely difficult-to-trace bugs.